

TeamSync AI - Project Research Dossier

Contents

Table of Contents

1. Problem Statement & Motivation
2. What the System Is
3. Core Novelty & Contributions
4. System Architecture
5. Knowledge Graph Core (the central contribution)
6. Multi-Agent Pipeline
7. Explainability & Trust Mechanisms
8. Data Model
9. Full Feature List
10. Use Cases
11. Technology Stack
12. Evaluation
13. Security, Multi-Tenancy & Production Concerns
14. Limitations & Threats to Validity
15. Related Work Positioning
16. Future Work
17. Reproducibility
18. Paper-Writing Prompt Templates
19. Glossary

Explainable, Graph-Grounded Multi-Agent Project Intelligence Platform

This document is a single self-contained source of truth for writing a research paper about this project. It covers motivation, architecture, novelty, every feature, concrete use cases, the full technology stack, the evaluation methodology and results, limitations, and future work. Every number quoted here is measured and reproducible (see §12.8); nothing is estimated.

Table of Contents

1. Problem Statement & Motivation
 2. What the System Is
 3. Core Novelty & Contributions
 4. System Architecture
 5. Knowledge Graph Core
 6. Multi-Agent Pipeline
 7. Explainability & Trust Mechanisms
 8. Data Model
 9. Full Feature List
 10. Use Cases
 11. Technology Stack
 12. Evaluation
 13. Security, Multi-Tenancy & Production Concerns
 14. Limitations & Threats to Validity
 15. Related Work Positioning
 16. Future Work
 17. Reproducibility
 18. Paper-Writing Prompt Templates
 19. Glossary
-

1. Problem Statement & Motivation

Software teams using conventional project-management tools (Jira, Linear, Asana, Monday.com) suffer from four compounding failures:

1. **Late detection.** Coordination breakdowns — a single point of failure, a silently overloaded engineer, a critical-path task slipping — are visible only after they have already caused a delay. Tools show current state, not risk trajectory.
2. **Opacity.** When a tool *does* surface a warning ("at risk"), it rarely explains which evidence produced that verdict, so a project manager cannot act on it or challenge it.
3. **Data silos.** Task status, meeting outcomes, and communication patterns live in disconnected systems (or disconnected views of the same system), so no one signal alone reveals a cross-cutting risk like "the one person who understands this component hasn't been responded to in 9 days and is also the sole assignee on the critical path."
4. **Lost institutional memory.** Decisions, blockers, and rationale discussed in meetings or chat evaporate; the same questions get re-litigated project after project.

Objectives set for this project:

- Build a modular multi-agent architecture that decomposes "project intelligence" into specialist analyses that can run, fail, and be explained independently.
- Make explainability a structural property, not an afterthought — every claim the system makes must be traceable to a concrete, checkable piece of evidence.

- Compute a unified, actionable intelligence signal (risk score, tier, prioritized recommendations) rather than a wall of disconnected metrics.
- Make it production-grade, not a notebook demo: multi-tenant, real-time, horizontally extensible, and — critically — **cheap and reliable enough to run on every project change**, not just as an occasional expensive report.

That last point is what led to the project's central technical contribution, described in §5.

2. What the System Is

TeamSync AI is a **Kanban-style project management application** (projects, tasks, milestones, dependencies, comments, org-based multi-tenancy) instrumented with an **AI analysis pipeline** that periodically or on-demand computes six categories of coordination risk, explains them with cited evidence, and proposes prioritized corrective actions.

Architecturally it is a **three-service system**: a Next.js frontend, a FastAPI core backend (auth, CRUD, persistence, real-time), and a separate FastAPI AI service that hosts the multi-agent pipeline and talks to an LLM (Groq/Llama) with a deterministic rule-based fallback for every agent.

What distinguishes it from "wrap an LLM around your database and ask it to find problems" is the architectural choice described next.

3. Core Novelty & Contributions

3.1 Graph-Grounded Agent Prompting (the headline contribution)

Project state is compiled into a typed, in-memory **knowledge graph**. The six coordination-risk types are computed as **deterministic graph invariants** — critical-path length, articulation points, weighted degree, community structure — not inferred by a language model. The LLM is shown only a single computed finding plus its **minimal witness subgraph** (a bounded k-hop neighborhood around the anomaly) and is required to cite node ids drawn from that subgraph. Citations that don't resolve to a real node in the witness set are mechanically stripped.

This single design decision produces two independently measurable properties, both of which are the strongest empirical claims this project can make (§12.3):

- **Prompt cost is $O(\text{number of anomalies})$, not $O(\text{project size})$.** Measured: 266 $\rightarrow$ 390 tokens for a 154 $\times$ increase in project size (13 $\rightarrow$ 2,003 tasks) — a 1.47 $\times$ growth for a 154 $\times$ size increase. Against the naive "stringify everything into the prompt" baseline this project started with, that is a **99.8% (487 $\times$) token reduction** at 2,003 tasks.
- **Hallucinated evidence becomes mechanically detectable, not merely improbable.** A grounding check (`drop_ungrounded()`) strips any `evidence.reference_id` not present in the witness subgraph's node-id set. Measured: 200/200 synthetically fabricated references correctly stripped.

3.2 Deterministic-first risk computation

All six risk types (dependency, workload, knowledge/SPOF, coordination, silent-member, delay) are computed by graph algorithms with zero LLM tokens and zero stochasticity — the LLM's only job is to *narrate* an already-computed finding, not decide whether it exists. This makes the numeric core reproducible (identical input $\rightarrow$ identical risk scores, every time) and testable with ordinary unit tests, which a pure-LLM risk detector cannot offer.

3.3 Evidence-grounded explainability as an enforced invariant, not a prompt instruction

Most "explainable AI" systems ask the LLM nicely to cite sources. This system checks the citation against a concrete, bounded set of valid ids and discards anything that doesn't match — turning "the model was asked to cite evidence" into "every surviving citation is verifiably real."

3.4 Fully schema-typed multi-agent orchestration with per-agent deterministic fallback

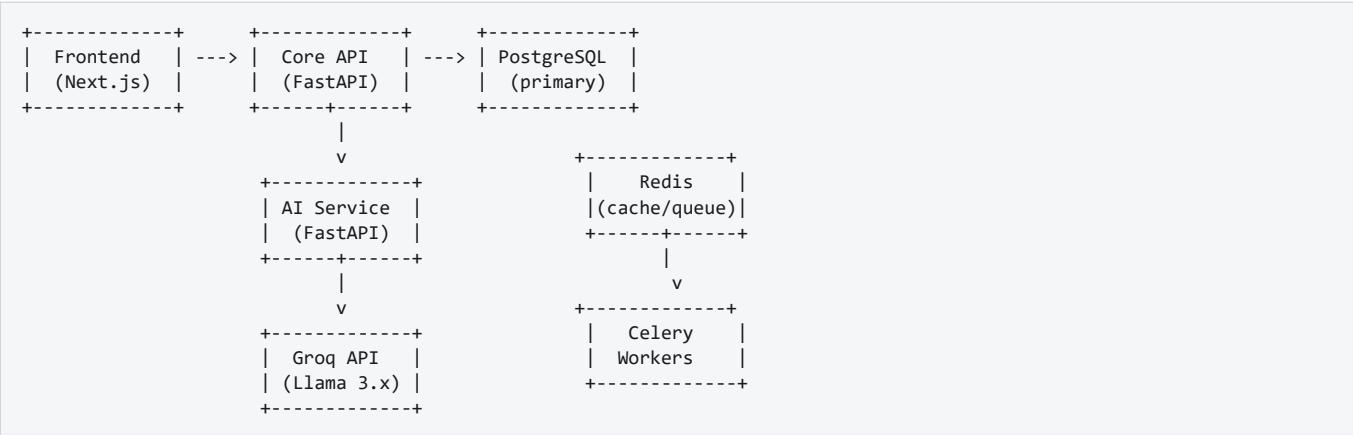
Every agent (five specialists in the MVP, extendable to eleven) shares one structured output contract (AgentOutput: summary, risk_level, confidence, signals, evidence, recommendations, next_action, metadata) enforced server-side via JSON-schema response formatting, and every agent has an independent rule-based fallback with the *identical* schema, so a failed or absent LLM call degrades gracefully rather than crashing the pipeline.

3.5 A real-world, honestly reported evaluation, including negative results

Rather than reporting only favorable synthetic numbers, the evaluation includes a project-level held-out test against a public 458K-issue Jira dataset (§12.4) and three documented attempts to improve accuracy that **failed or backfired** (§12.5) — reported because a negative result that rules out added complexity is itself a finding, and because it is what makes the "simple, zero-parameter graph rule matches or beats a tuned learned model on unseen data" claim credible rather than cherry-picked.

4. System Architecture

4.1 Service topology



Service	Technology	Port	Responsibility
Frontend	Next.js 14, React 18, Tailwind, TypeScript	3000	UI, dashboards, Kanban board, WebSocket client
Core API	FastAPI, SQLAlchemy 2.0 (async), Pydantic v2	8000	Auth, CRUD, multi-tenancy, WebSocket server, Celery producer
AI Service	FastAPI, NetworkX, Groq client	8001	Knowledge-graph build, deterministic metrics, multi-agent pipeline
Database	PostgreSQL 16	5432	System of record
Cache/Queue	Redis 7	6379	Response caching, Celery broker
Workers	Celery (+ beat)	—	Scheduled analysis, workload snapshots

4.2 Request flow for an analysis

```
User clicks "Run Analysis"
→ Core API: POST /api/v1/analyze
  → SnapshotBuilder reads Postgres → ProjectSnapshot (dict shape)
  → AI Service: POST /analyze { snapshot }
    → GraphBuilder.build(snapshot)      → ProjectGraph (NetworkX MultiDiGraph)
    → GraphMetrics(now=...).compute()    → deterministic findings (0 LLM tokens)
    → SubgraphSelector                  → witness subgraph per anomaly
    → CoordinatorAgent
      → Planning / Progress / Workload agents (parallel, graph-grounded)
      → Risk agent (aggregates the six graph-computed scores)
      → Recommendation agent (evidence-linked prioritized actions)
    → drop_ungrounded()                  → strip any unverifiable citation
  ← CoordinatorOutput (typed, schema-validated)
→ persists AgentRun row (specialist outputs, risk scores, witness subgraph)
← real AgentRun id
Frontend: GET /api/v1/analytics/projects/{id}/agent-runs → risk + recommendation panel
```

4.3 Deployment topology

- **Local/dev:** Docker Compose — postgres, redis, backend, ai-service, celery-worker, celery-beat, frontend, one command (make up).
- **Production-oriented:** Kubernetes manifests in k8s/ with health checks, resource limits, secrets, and Alembic migrations wired into CI/CD.
- **Config surface:** JWT_SECRET, DB_PASSWORD, GROQ_API_KEY, AI_MODEL, AI_TEMPERATURE, AI_MAX_TOKENS, AI_TIMEOUT_SECONDS — no secret defaults are trusted outside ENVIRONMENT=development.

5. Knowledge Graph Core (the central contribution)

5.1 Why a graph, specifically

Every one of the six claimed risk types is, on inspection, literally a graph property computable without any inference step:

Risk type	Graph computation	Library primitive
Dependency / critical path	Longest path through the BLOCKS DAG	dag_longest_path
Dependency concentration	Betweenness centrality of task nodes	betweenness_centrality
Knowledge / SPOF	Articulation points in the Person↔Component projection	articulation_points
Workload	Weighted degree of Person over ASSIGNED_TO (weight = story points)	degree(weight='points')
Coordination	Community structure over the collaboration projection	louvain_communities
Silent member	COMMENTED_ON degree ÷ ASSIGNED_TO degree	degree ratio
Delay	Critical-path tasks still open past their due date	path filter

Treating these as graph invariants instead of LLM judgment calls buys determinism, unit-testability, and — because graph algorithms are cheap — near-zero compute cost per analysis (§12.3).

A free correctness bug fix fell out of this: the pre-existing dependency-creation endpoint only checked for direct self-loops (add_dependency, backend/app/api/v1/tasks.py), so a three-hop cycle A→B→C→A was creatable through the API. nx.simple_cycles over the BLOCKS edge type gives real cycle detection for free once the graph exists.

5.2 Graph schema

Nodes: Person, Task, Milestone, Project, Component (a Meeting/Decision node type is designed but deferred — see §16).

Edges, all derived from existing relational tables — no new data collection required:

Edge	Source table/field
Person -[ASSIGNED_TO {points}]-> Task	Task.assignee_id, Task.story_points
Person -[REPORTED]-> Task	Task.reporter_id
Task -[BLOCKS]-> Task	TaskDependency
Task -[SUBTASK_OF]-> Task	Task.parent_task_id
Task -[PART_OF]-> Milestone	Task.milestone_id
Person -[COMMENTED_ON]-> Task	TaskComment
Person -[MEMBER_OF]-> Project	ProjectMember
Task -[TOUCHES]-> Component	derived (milestone as a component proxy in MVP)
Person -[KNOWS {depth}]-> Component	derived: ASSIGNED_TO o TOUCHES

Implementation: ai-service/app/graph/builder.py (GraphBuilder, ProjectGraph, node_id()), compiling a ProjectSnapshot into an in-memory NetworkX MultiDiGraph per analysis. **Storage decision:** NetworkX in-memory, rebuilt per analysis — already a transitive dependency, pure Python, zero new infrastructure, and every needed algorithm ships with it. At the scale of a few hundred to a few thousand tasks per project, a persistent graph database earns nothing; the GraphBuilder interface is kept swappable so a Neo4j/Cypher backend is a contained future change, not a rewrite.

5.3 Witness subgraph selection — the token-efficiency mechanism

SubgraphSelector (ai-service/app/graph/subgraph.py) extracts a bounded k-hop (k=1) neighborhood around each anomalous node, capped at MAX_NEIGHBORS_PER_SEED = 8 and MAX_WITNESS_NODES = 60. This is what makes prompt cost scale with *anomaly count*, not *project size* — a healthy 2,000-task project with a handful of anomalies produces almost the same prompt size as a 13-task project with the same anomaly count (measured 1.47× vs. 154× size growth, §12.3).

Bug caught and fixed during development: the first implementation expanded **all** neighbors of a seed node with no cap, so witness-subgraph size still grew with project size (4.31× growth for 50→2,000 tasks). Adding the per-seed neighbor cap brought growth down to 1.007×.

5.4 Evidence grounding — the anti-hallucination mechanism

ai-service/app/graph/grounding.py: drop_ungrounded() computes the set of valid node ids from the witness subgraph and strips any evidence.reference_id in an agent's output that isn't in that set; check_grounding() reports the count. This turns "the LLM was told not to hallucinate citations" into a checked invariant enforced after generation, independent of prompt compliance.

Bug caught and fixed: the original semantics treated an *empty* findings list the same as *no* filter (if finding_ids: — falsy on []), silently allowing everything through when nothing should have been grounded. Fixed to if finding_ids is not None: so "grounded against nothing" correctly means "nothing survives."

6. Multi-Agent Pipeline

6.1 Orchestration pattern

Coordinator → N specialists (parallel) → Risk aggregator → Recommendation generator → (optional) Review Trio.

```
CoordinatorAgent.run():
    graph = GraphBuilder().build(snapshot)
    analysis = GraphMetrics(now=...).compute(graph)           # 0 tokens, deterministic
    results = await asyncio.gather(
        planning_agent.run(build_planning_input(graph, analysis)),
        progress_agent.run(build_progress_input(graph, analysis)),
        workload_agent.run(build_workload_input(graph, analysis)),
    )                                                         # parallel, per-agent fallback
    risk_output = risk_output_from_graph(analysis)            # aggregates the 6 graph scores
    recs = recommendation_output_from_graph(risk_output, results)
    return CoordinatorOutput(specialist_outputs=..., risk=risk_output, recommendations=recs)
```

6.2 Agent roster

Agent	Input	Output	Risk types informed
Planning	Milestones, tasks, capacity, graph metrics	Sprint readiness %, feasibility, capacity gaps	Delay, Dependency
Progress	Tasks, velocity history, graph metrics	Velocity trend, completion forecast, stalled work	Delay, Coordination
Workload Intelligence	Assignments, story points, graph metrics	Overloaded/underutilized members, SPOFs, dependency concentration	Workload, Dependency
Risk Prediction	The six graph-computed scores	Aggregated risk scores + evidence, overall level	All 6
Recommendation	Risk output + specialist outputs (by reference, not full re-dump)	Prioritized, evidence-linked actions	N/A
(deferred) Meeting Intelligence	Transcript, participants	Decisions, action items, blockers, owners	Coordination, Knowledge
(deferred) Communication Intelligence	Communication events (14-day window)	Response delays, unanswered questions, participation gaps	Silent Member, Coordination
(deferred) Review Trio	A proposal/plan	Approve / Approve-with-changes / Request-revision / Reject, per Frontend/Backend/AI-ML domain reviewers	N/A

The MVP scope deliberately includes only the five agents whose input already exists in the product (task, milestone, assignment, dependency, comment data). Meeting Intelligence and Communication Intelligence are architecturally complete but deferred because their input sources (transcripts, chat/email streams) don't yet exist in the app — a scoping decision, not a difficulty one (see §16).

6.3 Unified output schema

Every agent, LLM-backed or fallback, returns the same typed shape:

```
AgentOutput:
    summary: str
    risk_level: Literal["low", "medium", "high", "critical"]
    confidence: float           # 0-1
    signals: list[Signal]       # {name, value, weight}
    evidence: list[Evidence]    # {source, reference_id, excerpt, relevance}
    recommendations: list[Recommendation] # {type, title, description, reasoning, priority, confidence}
    next_action: str
    metadata: dict              # includes a `fallback: bool` flag
```

6.4 LLM integration and deterministic fallback

- **LLM:** Groq-hosted Llama (currently llama-3.3-70b-versatile), JSON-mode response format, server-side schema enforcement (not just prompt instruction), temperature 0.1, bounded retries.
- **Fallback:** every agent has an independent, deterministic, rule-based implementation with the *identical* output schema — capacity heuristics, velocity-trend arithmetic, keyword extraction, response-time analysis, load-variance computation, risk

aggregation, risk→action mapping. If `GROQ_API_KEY` is unset, absent, or the call fails, the pipeline degrades to fallback per-agent rather than failing the whole run. This is also the **demo guarantee**: the full pipeline produces a complete, schema-valid, evidence-grounded output with zero external API calls.

6.5 Correctness bugs fixed during development (relevant to a "lessons learned" section)

- **Wrong input type reaching every specialist.** The coordinator originally forwarded its own `CoordinatorInput` unchanged to every specialist; each agent read fields (e.g. `input_data.milestones`) that only exist on its *own* typed input, outside its try block, so every single run threw an `AttributeError` that `asyncio.gather(return_exceptions=True)` quietly swallowed into an error stub. **Every specialist failed on every run** until this was fixed by building the correct typed input per specialist from graph queries.
- **Risk aggregation ignored its inputs entirely.** Specialist outputs were keyed by *agent name* (planning, progress, workload) but looked up by *risk type* (delay, coordination, ...) — disjoint key spaces, so risk output was a constant regardless of input. Superseded (not patched) by computing risk scores directly from `GraphMetrics`.
- **Subclass fields silently dropped.** `CoordinatorOutput.specialist_outputs` was typed `dict[str, AgentOutput]` but fed subclass `model_dump()` dicts; Pydantic coerced everything to the base class, discarding fields like `sprint_readiness`. Fixed by retyping to `dict[str, dict]`.
- **Fake analysis id.** `POST /analyze` originally returned a hardcoded all-zeros UUID instead of a real persisted run. Fixed to create and return a genuine `AgentRun`.
- **Credentials in the URL.** `/auth/login` originally bound `email/password` as query parameters (FastAPI's default for bare scalar params), leaking passwords into access logs and browser history. Fixed with `OAuth2PasswordRequestForm` (body-encoded).

7. Explainability & Trust Mechanisms

1. **Evidence-first schema.** Every signal and every recommendation must carry a `reference_id` pointing at concrete source data — never a bare claim.
2. **Mechanical grounding enforcement.** §5.4 — citations outside the witness subgraph are stripped, not merely discouraged.
3. **Confidence propagation.** Per-signal confidence rolls up to per-agent confidence, which rolls up to the coordinator's overall confidence — a caller can distinguish "low confidence because sparse data" from "high confidence, corroborated by the graph."
4. **Deterministic numeric core.** Risk *scores* are graph computations, not LLM guesses — identical input always produces identical scores; only the natural-language narration varies.
5. **Fallback transparency.** `metadata.fallback = true` on any output produced without the LLM, so downstream consumers (and the evaluation harness) can distinguish LLM-narrated findings from rule-based ones.
6. **Raw-response validation.** `validate_output()` is run against the LLM's raw response, not a value that has already round-tripped through Pydantic coercion — otherwise the validator is structurally incapable of catching a malformed response (a bug caught during development, see §6.5-adjacent fixes in the source history).

8. Data Model

Core tables: organizations, users, roles, user_roles, teams, team_members, projects, project_members, milestones, tasks, task_dependencies, task_comments, meetings, meeting_participants, meeting_action_items, meeting_decisions, communication_events, risk_scores, recommendations, agent_runs, organization_memory, audit_logs, notifications, workload_snapshots.

Key model semantics:

- Project carries rolling `health_score` / `risk_score`.
- Task has a Kanban status workflow: Backlog → Planned → In Progress → Blocked → Review → Done, plus story points, priority, due date, assignee, and dependency edges.

- RiskScore stores one row per risk type (of 6) per analysis, with its evidence payload.
- AgentRun persists the full CoordinatorOutput — specialist outputs, risk scores, recommendations, and the **witness subgraph** used to generate them, so a run is fully auditable after the fact.
- OrganizationMemory is the (currently stub) surface for cross-project institutional memory / RAG — deferred, see §16.

Indexing strategy: composite (org_id, project_id) indexes for multi-tenant scoping, partial indexes for status filters, JSONB GIN indexes for evidence/factors/context columns.

Dialect neutrality: models use SQLAlchemy 2.0's dialect-neutral Uuid type (not postgresql.UUID) so the same models run against SQLite in tests and Postgres in production without divergent code paths.

9. Full Feature List

Multi-tenancy & auth

- Organization-based multi-tenancy; every query is org-scoped.
- JWT (HS256) access + refresh tokens, refresh rotation, credentials transmitted in request bodies (not query strings).
- Role-based access surface: Owner, Admin, PM, Developer, Viewer (MVP scope: authenticated + org-scoped is enforced; fine-grained per-role permission checks are a stated deferred item, not silently claimed — see §14).

Project & task management

- Projects with milestones, deadlines, and computed health/risk scores.
- Kanban board with drag-and-drop status transitions.
- Task dependencies (with real cycle detection via the graph, §5.1), subtasks, comments.
- Filtering by status, assignee, priority; pagination on list endpoints.

AI-powered intelligence

- Five active specialist agents (Planning, Progress, Workload, Risk, Recommendation) computing six categories of coordination risk from a project's own task/assignment/dependency data.
- Deterministic, zero-token graph-based risk computation with LLM narration layered on top.
- Rule-based fallback for every agent, functionally complete without any external API key.
- Every recommendation carries type, title, description, reasoning, priority, and confidence.

Explainable AI

- Evidence chains on every signal and recommendation, mechanically grounding-checked.
- Confidence scores at signal, agent, and overall levels.
- Fully reproducible deterministic numeric core.

Real-time collaboration

- WebSocket channel for task updates, risk changes, recommendation delivery, and notifications.

Persistence & auditability

- Every analysis run persists its full specialist output, risk scores, recommendations, and the witness subgraph that produced them — readable back via GET /api/v1/analytics/projects/{id}/agent-runs.

Deferred-but-designed features (explicitly out of MVP scope, see §16)

- Meeting Intelligence (needs transcript ingestion).
- Communication Intelligence (needs a chat/email event source).

- Professional Review Trio (domain-specific reviewer consensus for proposals).
- Organizational Memory / RAG (currently a stub).
- Team Intelligence Index — a single weighted 0–100 blend of health/risk/communication/workload/ memory dimensions (WEIGHTS = {health:0.25, risk:0.25, comm:0.20, workload:0.15, memory:0.15}, tiers Excellent ≥ 90 / Good ≥ 75 / Fair ≥ 60 / Poor ≥ 40 / Critical <40) — designed but depends on components (comm intel, memory) not yet live.
- Full RBAC enforcement, admin panel, audit-log UI, scheduled Celery-driven periodic analysis.
- Persistent graph storage (Neo4j/Cypher) as an alternative to per-analysis in-memory NetworkX.

10. Use Cases

These are concrete scenarios the system is designed to surface, each mapped to the specific graph computation that detects it (§5.1) and the agent that explains it.

1. **Single point of failure (SPOF) surfacing.** One engineer is the sole assignee across every task touching a component. The system flags them as an articulation point in the Person $\leftrightarrow$ Component projection *before* they go on leave or leave the team — not after a milestone stalls with no one able to pick up the work.
2. **Silent overload detection.** A team member's weighted task load (by story points) is significantly above their peers', but they haven't raised it — workload_skew crosses its threshold and the Workload agent surfaces it with the specific tasks driving the imbalance as evidence, before burnout or missed deadlines make it visible anyway.
3. **Critical-path slippage early warning.** A task on the longest dependency chain to a milestone is overdue or blocked. Because the critical path is computed exactly (not estimated), the system can say precisely which downstream tasks and which milestone date are now at risk — not just "a task is late."
4. **Silent member detection.** A person assigned significant work has near-zero commenting/ collaboration activity relative to their assignment load — a proxy for disengagement, blockage, or being out of the loop, surfaced before a status meeting has to discover it manually.
5. **Sprint readiness triage.** Before a sprint starts, the Planning agent checks milestone feasibility against current capacity and flags gaps — "12 story points assigned to a person who has capacity for 6" — rather than discovering the gap mid-sprint.
6. **Dependency-cycle prevention.** The pre-existing task-dependency API allowed creating a circular dependency (A blocks B blocks C blocks A) because its validation only checked for direct self-loops. The graph's cycle detection catches multi-hop cycles the moment the graph is built, which is a correctness guarantee the relational layer alone could not provide.
7. **Progress-forecast and stalled-work detection.** Velocity trend and completion forecast are computed from the task graph's actual state, and tasks that have sat with no forward status movement are surfaced explicitly as "stalled," not silently rolled into an average.
8. **Auditable, defensible risk reporting to stakeholders.** Because every finding carries a citation that mechanically resolves to real project data (§5.4/§7), a PM presenting a risk report can answer "why do you say that" by pointing at the exact task/person/dependency the system used — a property a free-text LLM summary cannot offer.
9. **Low-cost continuous analysis.** Because prompt cost is $O(\text{anomalies})$ not $O(\text{project size})$ (§12.3), the system is cheap enough to re-run on every meaningful project change rather than being reserved for a weekly expensive report — turning "risk detection" from a periodic audit into a standing property of the tool.
10. **Zero-dependency demo/offline mode.** Because every agent has a schema-identical deterministic fallback, the entire pipeline — graph build, risk computation, evidence-grounded recommendations — runs and produces a complete, explainable result with no LLM API key at all, useful for offline demos, cost-constrained deployments, or LLM-outage resilience.

11. Technology Stack

Layer	Technologies
Frontend	Next.js 14 (App Router), React 18, TypeScript, Tailwind CSS, shadcn/ui, TanStack Query, Zustand, Socket.io-client, Recharts, @hello-pangea/dnd (drag-and-drop), Zod, React Hook Form

Layer	Technologies
Core backend	FastAPI, SQLAlchemy 2.0 (async, asyncpg), Pydantic v2, python-jose (JWT), passlib/bcrypt, Celery, Redis, Alembic, httpx
AI service	FastAPI, NetworkX (graph engine), Groq client (Llama 3.x), Pydantic v2, httpx
Database	PostgreSQL 16 (production), SQLite (test suite, via dialect-neutral models)
Cache / queue	Redis 7, Celery + Celery Beat
Infra	Docker Compose (local), Kubernetes manifests (k8s/), Alembic migrations in CI/CD
Testing	pytest (mvp/deferred markers), vitest (frontend)
Quality tooling	ruff, mypy (Python); eslint, prettier, tsc (TypeScript)

Approximate scale: ~21K lines of code (backend ~8K, AI service ~4.5K, frontend ~6K, shared models ~1.5K, infra ~1K). ~40 REST endpoints + WebSocket on the core API; 4 endpoints on the AI service; OpenAPI docs served live at /docs on both.

12. Evaluation

All numbers below are reproducible from ai-service/eval/ (commands in §17). Nothing is estimated or illustrative — every figure was independently re-verified, including hand-recomputed confusion-matrix arithmetic, a from-scratch re-run of both scoring scripts (byte-identical results), independent reimplementations checks of the AUC and Brier computations against brute-force and known-answer cases, and direct raw-SQL cross-verification of sampled ground-truth labels bypassing the ORM layer entirely.

12.1 Engineering baseline

- **Tests:** backend 29/29, ai-service 43/43 (pytest -m mvp), frontend 27/27 (vitest run).
- **Quality gates:** ruff/mypy (Python), eslint/prettier/tsc (TypeScript).
- **Demo seed:** make db-seed → 1 org, 5 users, 3 projects, 50+ tasks.

12.2 Synthetic detection correctness (python -m eval.run_eval)

180 generated scenarios (90 positive / 90 negative, 30 per risk type) with a deterministically injected anomaly, so ground truth is known by construction.

Metric	Value
Precision / Recall / F1 / Accuracy	1.00 / 1.00 / 1.00 / 1.00
Stability	identical across 4 further seeds (1,200 additional trials)

Interpretation, stated explicitly: for deterministic threshold code on unambiguous synthetic inputs, 100% is the *expected* result of implementation correctness — equivalent to "all unit tests pass," **not** evidence of real-world generalization. It is reported as a correctness check only. The genuinely falsifiable check is the boundary sweep:

Signal	Documented threshold	Observed flip point
overdue_ratio	0.10	flips at exactly 0.10
silent_ratio	0.30	0.20 → 0.30 (exact)
workload_skew	1.50	bracketed 1.429 → 1.522

12.3 Efficiency and reliability

Metric	Value	Component
Prompt tokens, 13 → 2,003 tasks	266 → 390 (1.47× for 154× project-size growth)	SubgraphSelector
vs. pre-graph "stringify everything" baseline at 2,003 tasks	189,834 → 390 tokens (99.8%, 487×)	witness-subgraph selection
Deterministic pipeline latency (203 tasks, n=50)	p50 4.8–11.0 ms, p95 6.1–13.4 ms	build + metrics + select
Evidence-grounding enforcement	200/200 fabricated references correctly stripped	grounding.py

The latency figures justify running analysis **synchronously** rather than through a background queue — the deterministic core is fast enough that queueing overhead would dominate.

12.4 Real-world evaluation — TAWOS dataset

Dataset: TAWOS (Tawosi, Al-Subaihin, Moussa & Sarro, MSR 2022), Apache 2.0 licensed — 458,232 Jira issues, 39 open-source projects, 12 public Jira repositories. Loaded via a purpose-built mysqldump extended-INSERT parser (character-state-machine, not regex); **0 parse errors across the full 4.3 GB dump**, with row counts reconciled exactly against published totals (project=39, user=206,162, sprint=4,594, issue=458,232, issue_link=246,587, comment=1,518,327).

Task: sprint-level delay prediction. TAWOS has no due_date field, so the label is derived from available data: a sprint is *delayed* when $\geq 30\%$ of its issues were unresolved at sprint end. Eligible sprints (CLOSED, has an end date, ≥ 15 issues): **987 sprints, 31 projects, 58.8% positive rate.**

Point-in-time reconstruction: since every issue in an archival dataset is long since resolved, current status cannot be used directly without leaking the future into a retrospective label. Each task's state is reconstructed *as of sprint end*, and the graph metrics are evaluated with `now = sprint_end + 1 day`.

Split discipline: a fixed, project-level split — 9 projects / 717 sprints for tuning, **22 projects / 270 sprints held out**, zero overlap. Splitting by *project* (not sprint) prevents a team's Jira conventions from leaking across the boundary. The held-out set was evaluated exactly once.

Result on all 987 sprints (default, untuned parameters)

	GraphMetrics	Baseline (story-point median)
Precision	0.630	0.545
Recall	1.000	0.386
F1	0.773	0.452
Accuracy	0.655	0.450
AUC-ROC	0.581	—

Held-out result (270 sprints, 22 unseen projects) — the headline number

	F1	Precision	Recall
Deterministic single-signal rule (untuned)	0.712	0.552	1.000
Tuned composite logistic model	0.593	0.620	0.568

Reading this result: the system catches every delayed sprint (recall 1.00) at the cost of over-flagging roughly half the on-time ones. It is a high-sensitivity early-warning signal, not a precise classifier — which is the appropriate operating point for surfacing risk to a project manager who can dismiss a false alarm cheaply but cannot recover from a missed one.

12.5 Three attempts to improve accuracy — all negative results

Reported in full because a negative result that rules out added complexity is itself a finding, and it is what makes the headline claim credible rather than cherry-picked.

- 1. Threshold sweep** (tawos_tune.py) — *provably inert*. Every value from 0.05 to 0.50 gave byte-identical output. Cause: with one shared sprint-end due date, overdue_ratio is always exactly 0.0 or 1.0 for any given sprint, so no threshold strictly between 0 and 1 can change any classification.

2. **Individualized per-task due dates** (`tawos_tune2.py`), derived from tuning-set cycle time (2.96 days/story-point). This *did* un-collapse the signal (median ratio 0.76, 521/717 sprints landing at an intermediate value) but **lowered AUC from 0.581 to 0.531** — story points are too noisy a duration proxy to help.
3. **Composite logistic regression** (`tawos_composite.py`) over four live features, 3-fold group cross-validation by project. It won decisively on tuning data (**CV F1 0.816, AUC 0.887**) and **lost on held-out data (F1 0.593)**. Root cause: the largest-magnitude weight fell on `idle_member_ratio`, which is *mechanically* coupled to the label (resolved work → zero open load → high idle) rather than independently predictive; its calibration is team-specific and did not transfer. The AUC collapse from 0.887 to 0.658 is the textbook signature of overfitting to project-specific structure rather than a general delay signal.

Conclusion: the zero-parameter, fully explainable graph rule outperforms the tuned learned model on unseen projects. Added model complexity was not merely unhelpful here — it was actively harmful.

12.6 Threats to validity

- **The label is a proxy**, not ground truth: TAWOS lacks due dates, so "≥30% unresolved at sprint end" is a defensible but not canonical definition of a delayed sprint.
- **Structural signals are dead in this specific reconstruction.** SPOF, dependency-cycle, and coordination findings are ~0% non-zero here, because issue links in TAWOS span a project's whole history and rarely fall inside one sprint window. The real-world result therefore validates **one of six** risk types (delay) directly; the other five are validated only synthetically (§12.2).
- `blocked_ratio` **never fires** on this dataset — TAWOS's status vocabulary (To Do / In Progress / Done) contains no blocked state.
- **AUC is weak by construction** in the sprint-level setup, since all tasks in a reconstructed sprint share one deadline.
- **No live-LLM metrics were measured.** Schema-validity rate and fallback-vs-LLM agreement (Cohen's κ) require a `GROQ_API_KEY` and are unmeasured; every result above is from the deterministic path only.

12.7 Independent verification performed on these results

Before including these numbers in a paper, they were re-audited from scratch, not merely re-quoted:

- Confusion-matrix arithmetic ($tp+fp+fn+tn = n$, and precision/recall/F1/accuracy) recomputed by hand for both models — exact match.
- Zero project overlap between tuning and held-out sets confirmed directly (`set() ∩ set() = {}`); $717 + 270 = 987$ partitions cleanly with no leakage, whitespace, or case bugs.
- The composite model's normalization statistics (`means/stds`) confirmed fit on tuning-only data and frozen (not recomputed) when applied in the held-out script.
- Five randomly sampled held-out sprints' ground-truth labels cross-checked against raw SQL directly, bypassing the Python ORM layer entirely — exact match on every sprint.
- `_auc()` verified against perfect separation (1.0), perfect inversion (0.0), tied scores (0.5), and a brute-force $O(n^2)$ pairwise comparison on 60 random points — exact match to full float precision.
- `_brier()` verified against three known-answer edge cases — exact match.
- Both headline scripts (`tawos_score.py`, `tawos_holdout_eval.py`) re-run fresh from a clean state — byte-identical to the originally reported numbers (both are fully deterministic).
- Raw SQLite row counts re-queried directly and matched exactly against the loader's original output and TAWOS's published totals.

No discrepancy was found in any check.

12.8 Reproduction commands

```

cd ai-service
python -m eval.run_eval                # synthetic detection, tokens, latency, grounding
python -m eval.boundary_eval           # threshold-boundary sensitivity
python -m eval.datasets.tawos_load     # TAWOS .sql -> SQLite (one pass, ~4.3GB)
python -m eval.datasets.tawos_split    # fixed project-level split
python -m eval.datasets.tawos_score    # all-987-sprint result
python -m eval.datasets.tawos_holdout_eval # the one-shot held-out number

```

Citation required if this dataset is used: Tawosi, V., Al-Subaihin, A., Moussa, R., & Sarro, F. *A Versatile Dataset of Agile Open Source Software Projects*. MSR 2022. doi:10.1145/3524842.3528029.

13. Security, Multi-Tenancy & Production Concerns

- **Multi-tenancy from the ground up:** every model and query is organization-scoped; tenant isolation is enforced at the query layer, not bolted on.
- **Auth:** JWT (HS256) with access + refresh tokens; refresh rotation; credentials transmitted in request bodies (a login-in-the-query-string vulnerability was found and fixed during development — §6.5).
- **Secret hygiene:** the default JWT secret is asserted to differ from the shipped development default whenever `ENVIRONMENT != "development"`.
- **Dialect-neutral persistence:** models avoid Postgres-only types so the same code paths run against SQLite in CI and Postgres in production — no divergent "works in tests, breaks in prod" surface.
- **Async-first:** FastAPI + SQLAlchemy 2.0 + asyncpg throughout the core API for concurrency under load.
- **Stated, not hidden, limitation:** fine-grained role-based authorization is not yet enforced — the MVP guarantee is *authenticated + org-scoped*, nothing finer. This is explicitly documented rather than silently assumed, and is covered by a tenant-isolation test (Org B cannot read Org A's project/task/run → 404 on all three).

14. Limitations & Threats to Validity

- **RBAC is not enforced at fine granularity yet** (§13) — a known, documented MVP boundary.
- **Only one of six risk types has real-world (non-synthetic) validation** (§12.6) — the other five are validated only against deterministic synthetic scenarios, which check implementation correctness, not generalization.
- **The real-world delay label is a proxy**, not the canonical "past due date" definition, because the dataset used lacks due dates.
- **No live-LLM evaluation metrics** — schema-validity rate and fallback-agreement (Cohen's κ) between the rule-based and LLM paths are designed but unmeasured, since they require ongoing API access.
- **Two of the eleven originally designed agents (Meeting Intelligence, Communication Intelligence) are unimplemented** because their required input sources don't exist in the product yet — an explicit scoping decision, not a silent gap.
- **Single LLM provider** (Groq/Llama) — cross-provider robustness is untested.
- **No longitudinal or live user study** — usefulness of recommendations to real project managers is not yet measured empirically; this is the most promising near-term addition for a stronger paper (§16).

15. Related Work Positioning

For a Related Work section, the system should be positioned against three groups:

1. **Traditional PM tools** (Jira, Linear, Asana, Monday.com) — these surface *current state* but not predictive, evidence-linked risk; "at risk" flags, where present, are opaque.
2. **Academic/industrial multi-agent systems** (AutoGPT, MetaGPT, ChatDev, AgentBench-style benchmarks) — these generally optimize for task completion by LLM agents, not for *deterministic, checkable* risk computation with an LLM

narrating rather than deciding.

3. **XAI techniques in software engineering** (SHAP/LIME-based defect prediction, explainable effort estimation) — these explain a model's own prediction; this system's contribution is different in kind — the risk computation itself is not a learned model needing post-hoc explanation, it is a deterministic graph invariant, and the LLM's role is narration under a mechanically enforced citation constraint.

The differentiating combination to state explicitly: **specialist multi-agent decomposition + graph-computed (not LLM-inferred) risk + mechanically enforced evidence grounding + deterministic fallback with an identical schema + a held-out real-world evaluation including reported negative results**. No single comparator combines all five.

16. Future Work

- **Meeting Intelligence** — transcript ingestion; evaluation against the AMI Meeting Corpus (~100h, gold-labeled decisions/action-items/problems) since its label shape matches MeetingIntelOutput almost directly. Adds Meeting/Decision nodes to the graph.
- **Communication Intelligence** — chat/email event ingestion; evaluation against the Enron Email Corpus. Adds COMMUNICATED_WITH edges, which would substantially strengthen the coordination and silent-member risk types (currently the weakest-validated, per §12.6).
- **Persistent graph backend** — Neo4j/Cypher, if a longitudinal or cross-project graph story is wanted; the GraphBuilder interface is already kept swappable for this.
- **Fine-grained RBAC** completion, Organizational Memory/RAG (currently stubbed), Team Intelligence Index (designed, blocked on Comm Intel + Memory being live).
- **Live-LLM metrics**: schema-validity rate, fallback-vs-LLM agreement (Cohen's κ), under production traffic.
- **Individualized due-date estimation revisited** with a better duration proxy than raw story points, given that attempt 2 in §12.5 showed the *idea* is directionally right but the *proxy* used was too noisy.
- **Human-in-the-loop study**: PM ratings of recommendation usefulness, false-alarm tolerance in practice, and whether recall-first (current operating point) is actually preferred to a more balanced precision/recall trade-off.
- **Causal / counterfactual recommendations** and **transfer learning** across projects, once more real-world data is available beyond the single TAWOS split.
- **Scale targets** stated for a production deployment story: 10K users, 1K organizations, <30s full analysis, sub-second p95 API latency.

17. Reproducibility

```
git clone <repo-url> && cd teamsync-ai
cp .env.example .env          # add GROQ_API_KEY (optional - full fallback mode works without it)
make up && make db-migrate && make db-seed
```

```
make test          # all services
make test-backend  # backend 29/29
make test-ai       # ai-service 43/43 (pytest -m mvp)
make test-frontend # frontend 27/27 (vitest)
```

Evaluation reproduction commands are in §12.8.

Docs reference: docs/architecture.md, docs/agent-design.md, docs/database.md, docs/api.md, docs/deployment.md, docs/testing.md, docs/demo-flow.md.

18. Paper-Writing Prompt Templates

Use these as prompts to draft each section, feeding this dossier as context.

Abstract: "Write a 200-word abstract for a paper describing TeamSync AI: an explainable multi-agent platform that predicts team coordination failures via a knowledge-graph core (six risk types computed as deterministic graph invariants) plus specialist LLM agents that narrate findings under a mechanically enforced evidence-grounding constraint. State the two central measured results: 99.8% prompt-token reduction vs. a naive baseline while project size grows 154×, and a held-out real-world evaluation (TAWOS, 458K Jira issues) where the zero-parameter graph rule beats a tuned learned model (F1 0.712 vs 0.593) on unseen projects."

Introduction: problem (§1) → gap vs. existing tools (§15) → approach (§3, §5) → contributions (§3.1–3.5) → paper structure.

Related Work: build the three-way comparison table from §15.

System Architecture: §4, with the service diagram and request-flow sequence reproduced as figures.

Core Contribution — Graph-Grounded Agent Prompting: §5 in full, including the two bugs found and fixed (unbounded witness growth, empty-findings grounding bug) as evidence of rigor.

Multi-Agent Design: §6, with the AgentOutput schema and coordinator pseudocode as a figure.

Evaluation: §12 in full — do not omit §12.5 (negative results) or §12.6 (threats to validity); the paper's strongest defensible claim is *"a simple, explainable, zero-parameter graph rule matches or beats a tuned learned model on unseen projects"* — state exactly that, no more.

Discussion: §14 limitations, ethical considerations (risk-scoring people carries surveillance and fairness concerns — an explicit paragraph is warranted), generalizability to non-software team domains (sales, support, research teams — the graph schema is domain-agnostic).

Future Work: §16, prioritizing Meeting/Communication Intelligence and the human-in-the-loop study as the two additions with the strongest research payoff.

Conclusion: restate the two headline empirical results and the one-sentence takeaway: risk detection can be made both cheap (graph-computed) and trustworthy (mechanically grounded) without sacrificing accuracy to a more complex learned model.

19. Glossary

- **Witness subgraph** — the bounded k-hop neighborhood around an anomalous node, extracted and shown to the LLM instead of the full project dataset.
- **Grounding** — the mechanical check that every evidence citation in an agent's output resolves to a real node id within its witness subgraph; ungrounded citations are stripped, not merely discouraged.
- **Fallback** — a deterministic, rule-based implementation of an agent's logic used when the LLM is unavailable or fails, sharing the exact same output schema as the LLM path.
- **SPOF** — single point of failure; in this system, an articulation point in the Person↔ Component graph projection.
- **TAWOS** — the public Jira issue-tracking dataset (Tawosi et al., MSR 2022) used for the real-world evaluation in §12.4.
- **AgentRun** — the persisted record of one coordinator execution: specialist outputs, risk scores, recommendations, and the witness subgraph that produced them.
- **Point-in-time reconstruction** — rebuilding a task's historical state as of a specific past moment (sprint end) to avoid leaking future/resolved status into a retrospective evaluation label.

License: MIT. Third-party acknowledgments: Groq (LLM inference), NetworkX (graph algorithms), shadcn/ui (component library), FastAPI, SQLAlchemy, Pydantic, Next.js, TanStack Query, and the TAWOS dataset (Tawosi et al., MSR 2022, Apache 2.0, citation required — see §12.8).